Московский физико-технический институт

Проект
по дисциплине
«Продуктовые решения в облаке»
Часть 1.

Выполнили:
Белоусова Екатерина Александровна
№101824

Москва, 2026

Оглавление

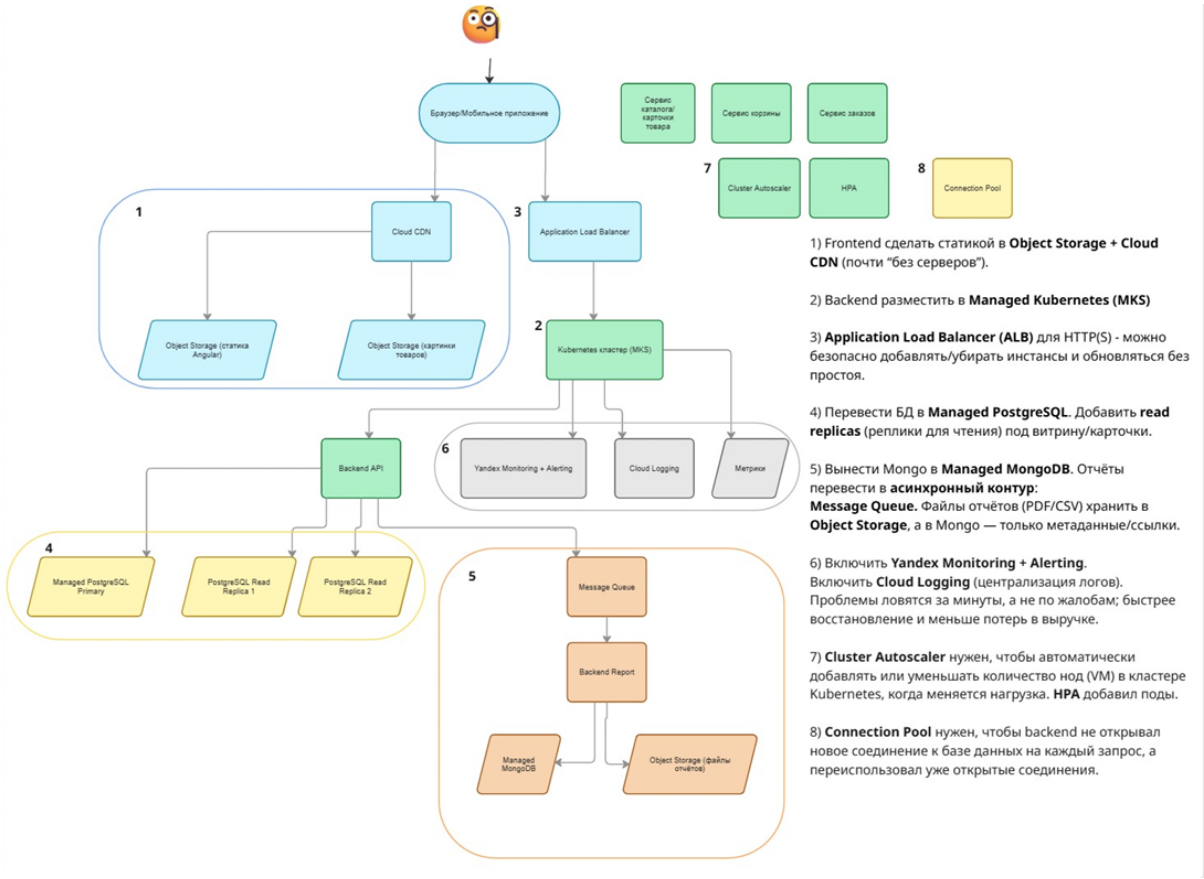
Оглавление **2**
1. Узкие места текущей схемы **3**
2. Предлагаемые решения **4**
1) Frontend как статика в Object Storage + Cloud CDN 4
2) Backend в Managed Kubernetes (MKS) 5
3) Application Load Balancer (ALB) для HTTP(S) 5
4) Managed PostgreSQL + read replicas под витрину/карточки 6
5) Managed MongoDB + отчёты в асинхронный контур (Message Queue + Object Storage) 6
6) Monitoring + Alerting и Cloud Logging 7
7) Cluster Autoscaler и HPA 7
8) Connection Pool 8
Заключение **9**

1. Узкие места текущей схемы

Узкие места	Влияние на UX	Рекомендации по решению
1 инстанс frontend+backend → нет отказоустойчивости и горизонтального масштабирования	- если сервер “упал” или завис — магазин недоступен; - если людей стало больше — сервер не успевает → страницы грузятся медленно, появляются ошибки.	1) Frontend сделать статикой в Object Storage + Cloud CDN (почти “без серверов”). 2) Backend разместить в Managed Kubernetes (MKS) или Instance Group (несколько копий).
нет балансировщика → нельзя просто добавить вторую/третью копию приложения и распределить трафик.	- в сезон нагрузка растёт, а “прибавить мощности” сложно и рискованно → подвисания, 5xx ошибки; - любые работы/обновления чаще приводят к простоям.	3) Application Load Balancer (ALB) для HTTP(S) можно безопасно добавлять/убирать инстансы и обновляться без простоя.
PostgreSQL 1 инстанс, профиль “много SELECT” → нужен read-scaling. Это способ увеличить пропускную способность **на чтение**	- карточка товара открывается медленно (характеристики/цены/остатки подгружаются долго); - в пики возможны таймауты (“не удалось загрузить данные”, “попробуйте позже”).	Перевести БД в **Managed PostgreSQL**. Добавить **read replicas** (реплики для чтения) под витрину/карточки. В приложении разделить: чтение каталога → реплики записи/заказы → primary
MongoDB на одном сервере с PostgreSQL → отчёты “съедают” CPU и	- во время отчётов вдруг тормозит весь магазин: каталог/карточки/корзина отвечают хуже; - пользователи видят “внезапные просадки” скорости без понятной	Вынести Mongo в **Managed MongoDB** (отдельный кластер). Отчёты перевести в **асинхронный контур**: **Message Queue** —

влияют на транзакционную БД	причины.	очередь задач отчётов backend-report как воркеры (в MKS или **Serverless Containers**) Файлы отчётов (PDF/CSV) хранить в **Object Storage**, а в Mongo — только метаданные/ссылки.
NFS на HDD, 100 img/s → упор в I/O → нужно вынести картинки в объектное хранилище + CDN (сервер NFS становится **единой точкой узкого места**: диск “захлёбывается”, растёт время ответа)	- картинки грузятся долго или не грузятся;\ - страница выглядит “сломанной”: текст есть, а изображения появляются через секунды → ощущение, что всё тормозит; - в сезон это ухудшается кратно.	Object Storage
нет мониторинга/алертинга → ы становится известно о проблеме, когда уже жалуются пользователи/бизнес.	- бои длятся дольше, потому что их поздно обнаруживают; - проблемы повторяются (нет данных “почему”), а пользователи сталкиваются с нестабильностью: то быстро, то медленно, то ошибки.	Yandex Monitoring + Alerting

2. Предлагаемые решения



Ссылка на схему: https://drive.google.com/file/d/1fQ_PAvV4nua0mg-VtXcLX3V5Uk6i13qL/view?usp=sharing

1) Frontend как статика в Object Storage + Cloud CDN

Как реализовано?	Что дает?
Карточка товара "ощущается" пользователем через скорость загрузки UI и картинок. Статика и картинки — самый массовый и "тяжёлый" трафик. **Object Storage** — специализированное хранилище для отдачи файлов, не упирается в "один диск/один сервер".	**Максимально быстрый первый экран и карточка товара** (меньше задержек, меньше "прыжков" интерфейса). **Автоматическое масштабирование** под сезонные пики (CDN и Object Storage рассчитаны на высокий параллелизм). Снижение стоимости и рисков: frontend почти "без серверов", меньше точек отказа.

Cloud CDN кэширует контент на edge: большая часть запросов обслуживается **без обращения к вашему backend**	

2) Backend в Managed Kubernetes (MKS)

Почему не serverless для backend	Почему MKS?	Что дает?
Для e-commerce критичны **стабильные p95/p99 задержки** на API карточки (каталог, цены, остатки). У serverless может быть **cold start**, и “первый запрос” иногда ощутимо медленнее. Это прямо бьёт по конверсии	Можно держать backend **постоянно прогретым** и масштабировать горизонтально копиями. MKS удобен, если сервисов будет больше (витрина/корзина/заказы/платежи), есть HPA и изоляция по ресурсам.	**Предсказуемая скорость API** (без скачков на старте). **Горизонтальное масштабирование** (добавили копии → выдержали сезон). **Отказоустойчивость**: один инстанс упал — остальные продолжают обслуживать.

3) Application Load Balancer (ALB) для HTTP(S)

Почему ALB обязателен?	Что это дает?
Когда backend работает в нескольких копиях, нужен “умный вход”, который: ○ распределяет трафик, ○ проверяет здоровье инстансов, ○ убирает “больные” узлы из выдачи, ○ поддерживает безопасные обновления.	**Обновления без простоя** (постепенно). **Масштабирование без риска**: добавили/убрали копии — пользователи не заметили. Единый контролируемый вход с TLS, логами, метриками, правилами маршрутизации.

4) Managed PostgreSQL + read replicas под витрину/карточки

Почему это важно?	Что решают read replicas?	Что это даёт?
В интернет-магазине профиль нагрузки обычно “read-heavy”: каталог, карточки, фильтры → это масса **SELECT**. Один инстанс PostgreSQL становится бутылочным горлышком: CPU/IO/соединения растут → карточка начинает тормозить.	Реплики обслуживают **чтение** параллельно, а primary остаётся под записи (заказы, статусы). Это “правильный” способ масштабировать именно витрину и карточку без ломки транзакционного контура.	**Стабильная скорость карточки товара** в сезон. Меньше таймаутов, меньше “не удалось загрузить данные”. Плюс managed-сервис даёт HA, бэкапы, обслуживание без ручной рутины.

5) Managed MongoDB + отчёты в асинхронный контур (Message Queue + Object Storage)

Почему отчёты нельзя держать “в одном котле” с витриной	Почему асинхронно?	Что это даёт?
Генерация отчётов часто создаёт пики CPU/IO, а значит может **замедлить базу** и API карточки. Для e-commerce это критично: пользователи не	**Message Queue** отделяет “пик” запросов от реальной обработки: задачи выстраиваются в очередь, воркеры обрабатывают по мощности.	Отчёты **не влияют на витрину**. Сезонный рост по пользователям и рост по аналитике больше не

должны страдать из-за аналитических задач.	**Object Storage** — лучшее место для “тяжёлых файлов” (PDF/CSV): дешево, надёжно, масштабируется. В **MongoDB** держим только метаданные/статусы/ссылки — быстро и без раздувания.	конфликтуют. Можно масштабировать воркеры отчётов независимо.

6) Monitoring + Alerting и Cloud Logging

Почему это важно?	**Почему именно эти метрики?**	**Что это даёт?**
Без мониторинга вы узнаёте о проблеме **после жалоб**, когда уже потеряли конверсию/выручку. В e-commerce деградация часто начинается не с “падения”, а с роста задержек: страница “чуть медленнее” → конверсия падает.	**p95/p99 latency API карточки** — главный показатель пользовательского опыта. **5xx на ALB** — быстрый индикатор аварий/ошибок. **PostgreSQL connections/CPU + replication lag** — показывает, когда витрина упирается в базу или отстают реплики. **длина/возраст очереди отчётов** — контроль, что аналитика не “захлебнулась”.	Проблемы ловятся **за минуты**, а не “когда уже всё горит”. Быстрее восстановление (меньше потерь выручки), проще расследование причин (логи централизованы).

7) Cluster Autoscaler и HPA

Cluster Autoscaler нужен, чтобы автоматически добавлять или уменьшать количество нод (VM) в кластере Kubernetes, когда меняется нагрузка. HPA добавил поды.

8) Connection Pool

Connection Pool нужен, чтобы backend не открывал новое соединение к базе данных на каждый запрос, а переиспользовал уже открытые соединения.

Заключение

Предложенная архитектура специально оптимизирована под e-commerce: **всё, что отвечает за скорость карточки товара (статика/картинки/чтение каталога/API)** масштабируется отдельно и предсказуемо (CDN+Object Storage, backend в нескольких копиях за ALB, PostgreSQL с read-replicas). Тяжёлые фоновые задачи (отчёты) вынесены в **асинхронный контур**, чтобы никогда не “воровать” ресурсы у витрины. Наблюдаемость (Monitoring/Logging) делает качество сервиса управляемым: деградация ловится до того, как её заметит клиент.